



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

Subwords: lo que generalizan y lo que confunden

El mismo corte que le da vector a una palabra nueva puede acercar dos que no tienen nada que ver.

$\mathcal{G}_o = \{ \text{char n-gramas de } o \}$ el corte es ciego al significado

$v_o = \sum_{g \in \mathcal{G}_o} v_g$ hay vector aunque o no esté en el vocabulario

$\cos(v_o, v_{o'}) \sim |\mathcal{G}_o \cap \mathcal{G}_{o'}|$ sube con los subwords compartidos

comparten el lexema
el parecido es de verdad

		n-gr.	coseno
trabajo	trabajador	14	0,815
policía	policial	10	0,751
nación	nacional	6	0,304

comparten solo el final
el parecido es del corte

		n-gr.	coseno
viabilidad	debilidad	18	0,978
volando	blando	10	0,934
taxista	exista	10	0,911

el peor par de la derecha le gana al mejor de la izquierda
una terminación larga aporta más n-gramas que una raíz corta

Sirve para las OOV

«antiterrorismo» no está, y sale con
«terrorismo» de vecino.

Acierta con el lexema

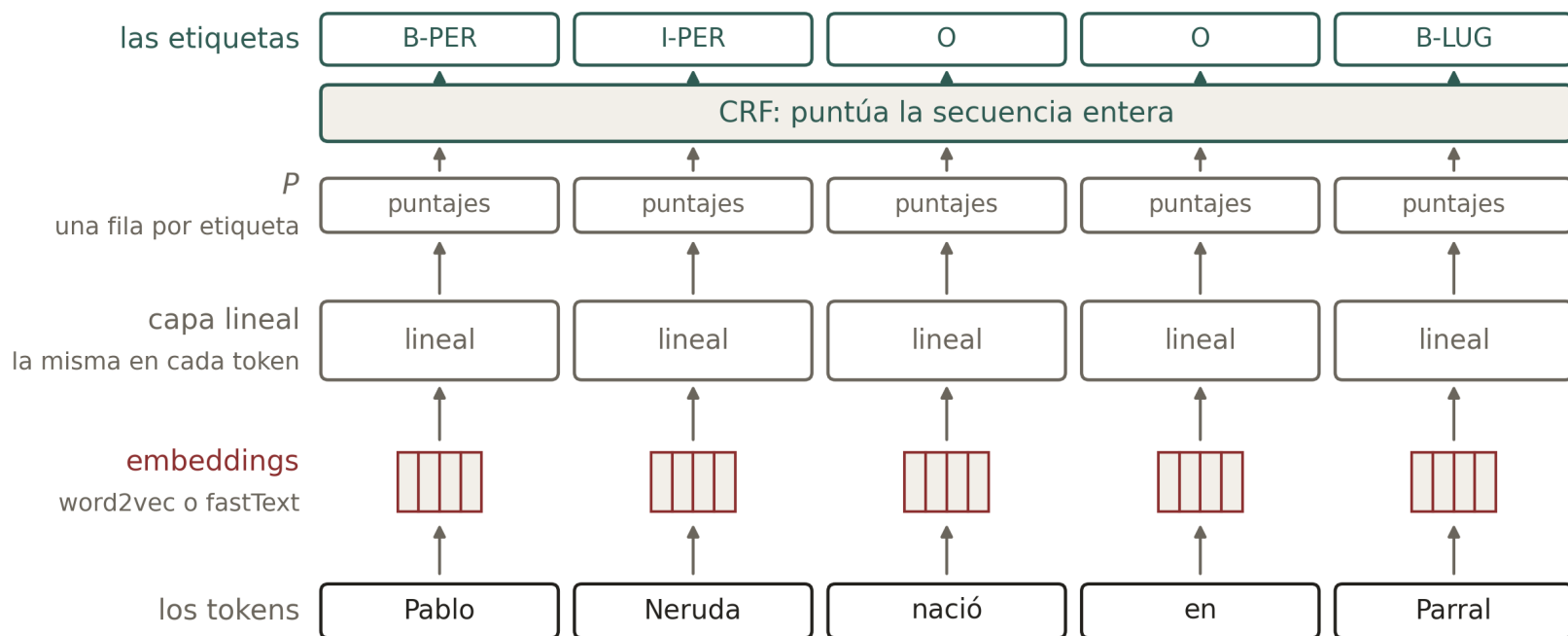
Si lo compartido es la raíz, el parecido
es real.

Falla con la terminación

Si es el sufijo, el parecido es ortográfico.

Cualquier encoder necesita un cabezal

Los vectores por token no resuelven nada. El cabezal decide la tarea, y para etiquetar secuencias una opción es el CRF.



El denominador del CRF

Es la misma softmax de siempre. Lo que cambia es sobre qué se suma en el denominador.

$$p(y_i=k \mid x) = \frac{e^{P_{i,k}}}{\sum_{j=1}^K e^{P_{i,j}}}$$

softmax por token: suma la columna

$$p(y \mid x) = \frac{e^{s(x,y)}}{Z(x)}$$

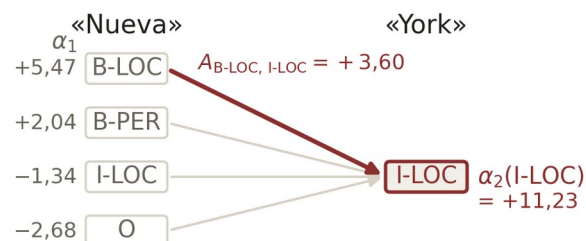
CRF: el denominador es Z

$$Z(x) = \sum_{k=1}^K \alpha_n(k)$$

lo que llegó a la última columna

$$\alpha_i(k) = \sum_{j=1}^K \alpha_{i-1}(j) e^{A_{j,k} + P_{i,k}}$$

cada nodo acumula lo que llega a él



y las otras cinco etiquetas

nueve aportes, uno por etiqueta anterior; el de B-LOC se lleva el 99,9 % del acumulado

Arriba el puntaje, abajo Z

La secuencia se normaliza con Z , no con la suma de una columna.

Alfa acumula lo que llega

Un aporte por cada etiqueta anterior, con su transición.

Se arrastra hacia adelante

Cada columna sale de la anterior: n por K números en total.

La cadena puede ir en una variable latente

El CRF pone la cadena en la etiqueta que se predice. El HMM la pone en un estado oculto que genera las palabras.

$$P(h_i | h_1, \dots, h_{i-1}) = P(h_i | h_{i-1})$$

la cadena es de Markov

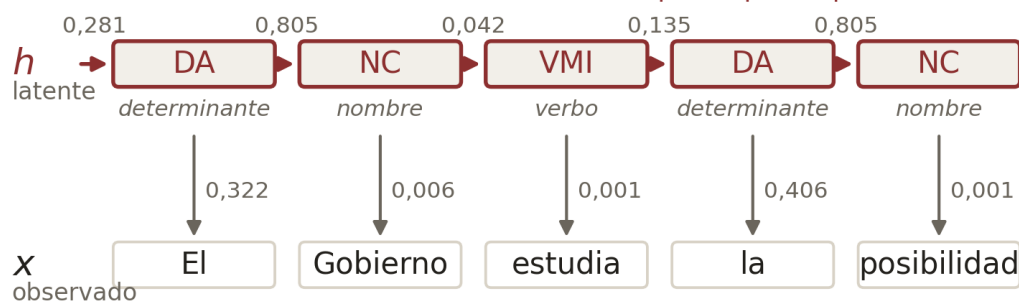
$$P(x_i | h_1, \dots, h_n) = P(x_i | h_i)$$

cada palabra sale de su estado

$$P(x, h) = \prod_{i=1}^n P(h_i | h_{i-1}) P(x_i | h_i)$$

la conjunta, no la condicional

la cadena va en la variable latente, no en la etiqueta que se predice



transiciones y emisiones medidas sobre conll2002

Dos suposiciones

El estado depende solo del anterior, y cada palabra sale solo de su estado.

Modela la conjunta

Explica cómo se produjo el texto, no solo cómo etiquetarlo.

Ojo con la notación

En el material clásico la cadena va en x . Aquí x es la entrada, siempre.

La tarea es POS tagging: las etiquetas son categorías gramaticales, no entidades como en las láminas anteriores.

Las h toman valores sobre las etiquetas

Cada h es una variable aleatoria sobre el conjunto de etiquetas. En entrenamiento las etiquetas se ven, así que las transiciones se cuentan.

$$h_i \in \{DA, NC, AQ, \dots\} \quad 59 \text{ valores posibles}$$

$$P(h_i=NC \mid h_{i-1}=DA) = \frac{c(DA, NC)}{c(DA)}$$

la etiqueta se observa

$$= \frac{20.677}{25.679} = 0,805$$

se cuenta y se divide

	h_i							
	DA	NC	AQ	SP	VMI	CC	RG	FC
DA	0,000	0,805	0,070	0,006	0,001	0,000	0,005	0,000
NC	0,009	0,020	0,198	0,336	0,042	0,047	0,010	0,126
AQ	0,011	0,195	0,050	0,274	0,036	0,060	0,010	0,142
SP	0,299	0,396	0,025	0,002	0,006	0,000	0,009	0,000
VMI	0,135	0,069	0,073	0,258	0,003	0,007	0,085	0,026
CC	0,157	0,184	0,110	0,137	0,089	0,000	0,043	0,017
RG	0,080	0,041	0,136	0,287	0,080	0,008	0,030	0,070
FC	0,095	0,112	0,063	0,172	0,142	0,060	0,038	0,000

las 8 etiquetas más frecuentes de las 59 del corpus contadas sobre conll2002, sin estimar nada

Una variable, 59 valores

h no es un número: toma valores en el conjunto de etiquetas.

Cada transición es una celda

P(h de i igual a NC dado h anterior igual a DA) es un número de una tabla.

Entrenar es contar

Con las etiquetas anotadas no hay que estimar nada iterativo.

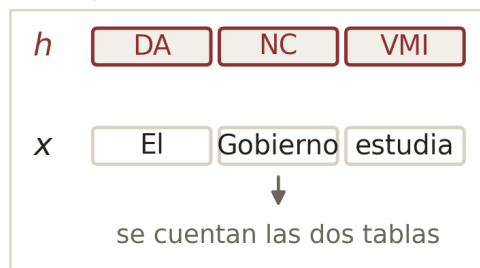
Entrenamiento e inferencia no ven lo mismo

En el corpus anotado la etiqueta se ve. Latente lo es recién cuando llega una oración sin etiquetar.

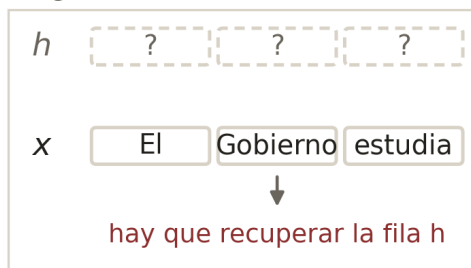
entrenamiento (x, h) $P(h_i | h_{i-1}), P(x_i | h_i)$ se cuentan y quedan fijas

inferencia x $h^* = \arg \max_h P(x, h)$ esto es lo que falta

entrenamiento
el corpus trae las dos filas



inferencia
llega la oración sola



«latente» lo es solo a la derecha: en el corpus anotado la etiqueta se ve

En entrenamiento están las dos

Se ven h y x a la vez, y por eso las tablas se cuentan.

Al final quedan dos tablas

Transiciones y emisiones, fijas: no se vuelven a tocar.

En inferencia falta la de arriba

Llega solo x , y hay que elegir la cadena h que mejor la explica.

Viterbi: qué es delta y para qué sirve

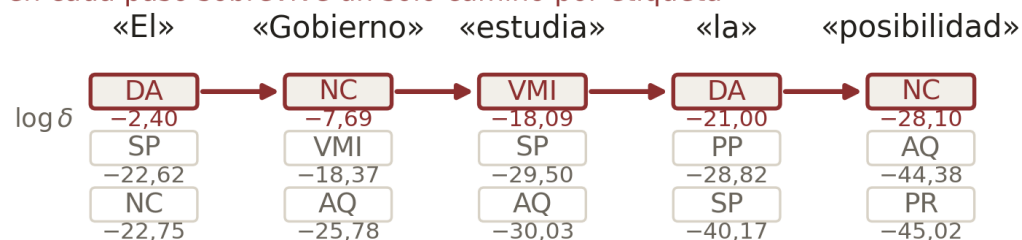
Delta es el puntaje del mejor camino que llega a una etiqueta usando solo las palabras vistas hasta ahí.

$$\delta_i(k) = \max_{h_{1:i-1}} P(x_{1:i}, h_{1:i-1}, h_i=k) \quad \text{el mejor camino a } k$$

$$= \max_j \delta_{i-1}(j) P(h_i=k \mid h_{i-1}=j) P(x_i \mid h_i=k) \quad \text{del paso anterior}$$

$$\max_k \delta_n(k) \quad \text{la mejor cadena}$$

en cada paso sobrevive un solo camino por etiqueta



bajo cada etiqueta, el log de su delta: el mejor camino que llega a ella baja al avanzar, porque cada paso multiplica una probabilidad más

se comparan dentro de la columna, nunca entre columnas

Un delta por casilla

Uno por posición y etiqueta: n por K números, no todas las cadenas.

Se calcula del anterior

Cada columna sale de la de su izquierda y de las tablas ya contadas.

Al final se lee el mayor

El mayor delta de la última columna es la mejor cadena completa.

De la cadena latente a la recurrente

En vez de codificar la secuencia en h y leerla como respuesta, se usa h para producir la respuesta.

HMM $h_i \in \{\text{DA}, \text{NC}, \dots\}$ $P(x_i | h_i)$ la palabra sale del estado

RNN $h_t \in \mathbb{R}^d$ $y_t = f(h_t)$ la salida sale del estado

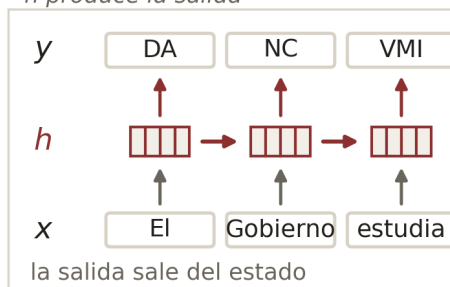
HMM

la secuencia va codificada en h



RNN

h produce la salida



h deja de ser la respuesta

En el HMM h es la etiqueta. En la RNN es un vector que no se muestra.

Y pasa a producirla

La salida sale del estado, no al revés: las flechas se dan vuelta.

De 59 valores a un vector

Deja de elegir entre etiquetas y pasa a ser cientos de números.

misma cadena, distinto papel: a la izquierda h es la respuesta, a la derecha la produce

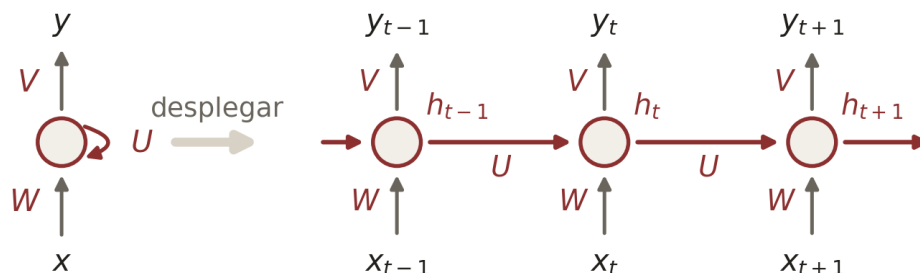
Las ecuaciones de transferencia

Un estado por posición, que mezcla la entrada de ahora con lo que traía. La salida sale de él por una capa lineal.

$$h_t = g(Wx_t + Uh_{t-1}) \quad \text{lo de ahora y lo que traía}$$

$$z_t = Vh_t \quad \text{la capa lineal: un puntaje por etiqueta}$$

$$y_t = \text{softmax}(z_t) \quad \text{la etiqueta de la posición } t$$



las tres matrices son las mismas en toda la secuencia: no hay una por posición

Tres matrices, no más

W para la entrada, U para el estado anterior, V para la salida.

Las mismas en toda la secuencia

No hay una matriz por posición: por eso sirve cualquier largo.

El cabezal es el mismo

Capa lineal y softmax, igual que sobre los embeddings estáticos.

La celda son capas densas

Dos productos matriz por vector, la suma de los dos vectores, y la activación. Eso es toda la recurrencia.

$$U\mathbf{h}_{t-1} \in \mathbb{R}^d$$

matriz por vector: da un vector

$$Wx_t \in \mathbb{R}^d$$

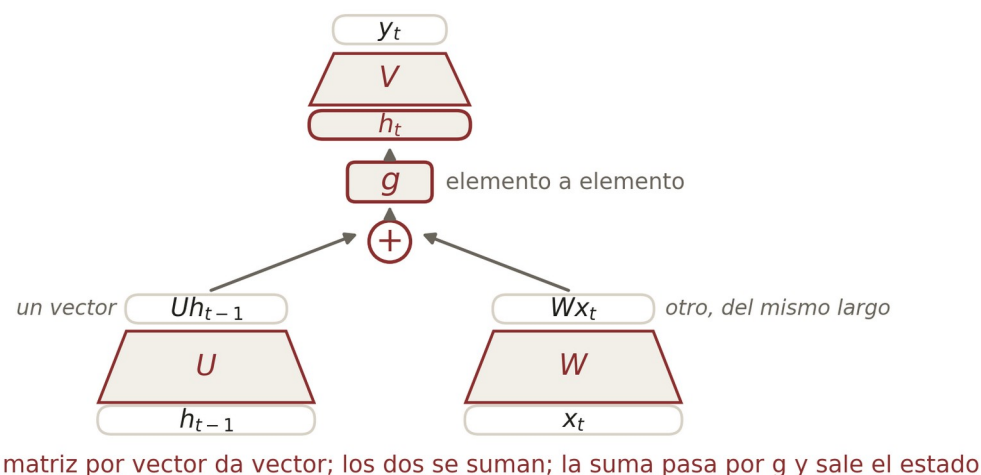
otra matriz, otro vector, mismo largo

$$\mathbf{h}_t = g(U\mathbf{h}_{t-1} + Wx_t)$$

se suman y la suma pasa por g

$$g = \tanh : \mathbb{R} \rightarrow (-1, 1)$$

elemento a elemento



Dos matrices, dos vectores

U sobre el estado anterior, W sobre la entrada, por separado.

Se suman y pasan por g

Los dos resultados tienen el mismo largo.

g es la activación

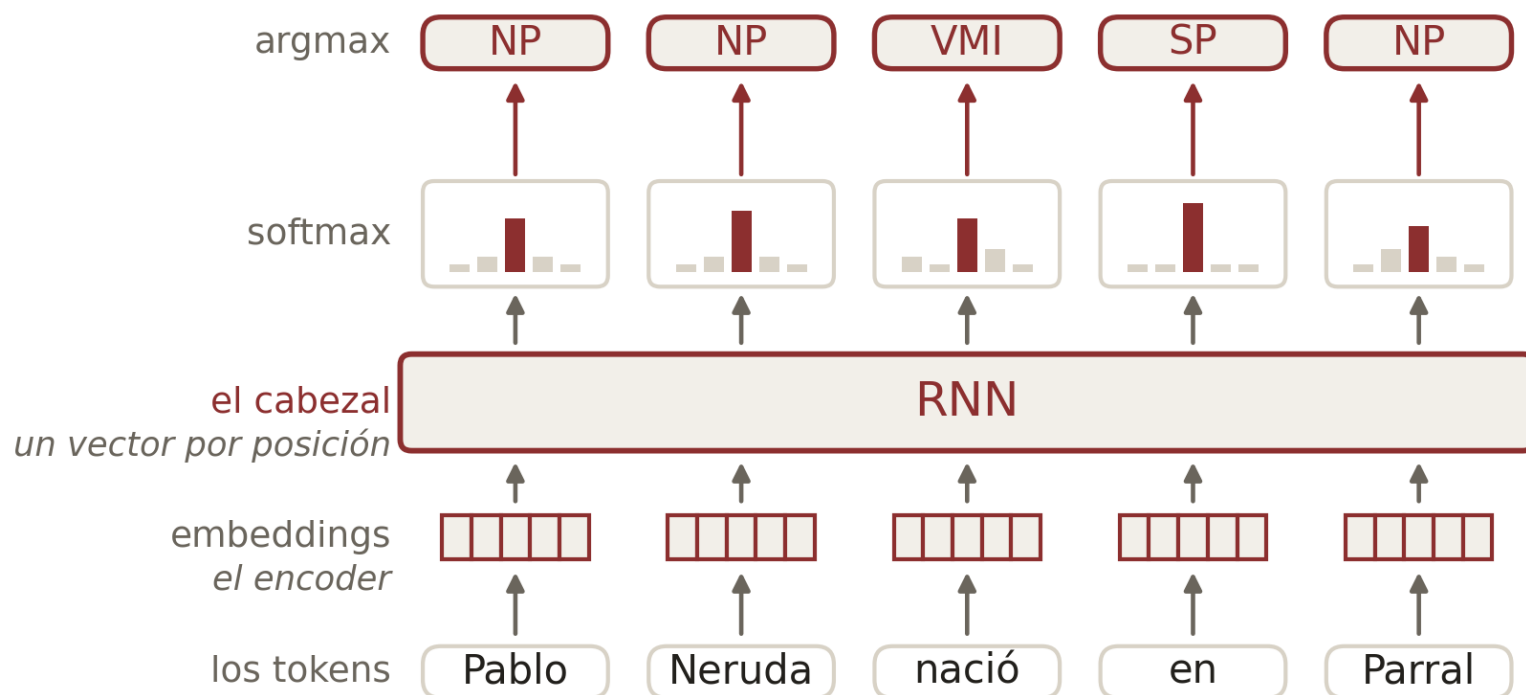
La misma familia de la clase 2, ahora por coordenada.

Por qué $\tanh$ y no sigmoide

Centrada en cero: la sigmoide sesga y degrada el gradiente.

La recurrente como cabezal

El encoder no cambió. Lo que se pone encima decide la tarea, y ahora arriba va una recurrente.



mismo encoder que con el CRF: lo que cambia es lo que se le pone encima

Hacia adelante no alcanza

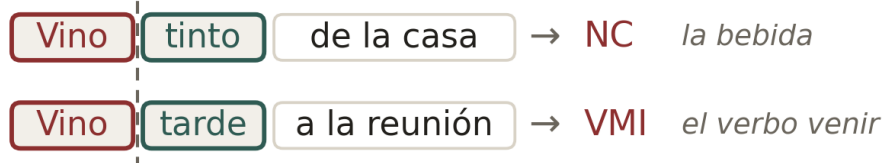
La lectura hacia adelante sigue el orden en que se lee. Para un encoder, lo que viene después también informa.

$$h_t^{(f)} = f(x_1, \dots, x_t) \quad \text{solo lo que ya se leyó}$$

$$h_t^{(b)} = f(x_t, \dots, x_n) \quad \text{lo que falta por leer}$$

$$h_1^{(f)} = f(x_1) \quad \text{en la primera posición, una palabra sola}$$

la misma palabra en la misma posición, con etiquetas distintas
lee hasta acá



hacia adelante, en la posición 1 el estado es idéntico en las dos
hacia atrás, ya leyó lo que las distingue

El estado solo ve el prefijo

En la posición t , la recurrente leyó de la primera palabra a esa.

A veces decide lo que sigue

«Vino» es la bebida o el verbo según la palabra siguiente.

Un encoder no tiene por qué leer en orden

La oración ya está entera: se puede recorrer también al revés.

Las ecuaciones de la bidireccional

Dos recurrentes independientes, una en cada sentido. Toda la diferencia está en el índice del estado que cada una toma.

$$h_t^{(f)} = g(W^{(f)}x_t + U^{(f)}h_{t-1}^{(f)})$$

lee hacia adelante

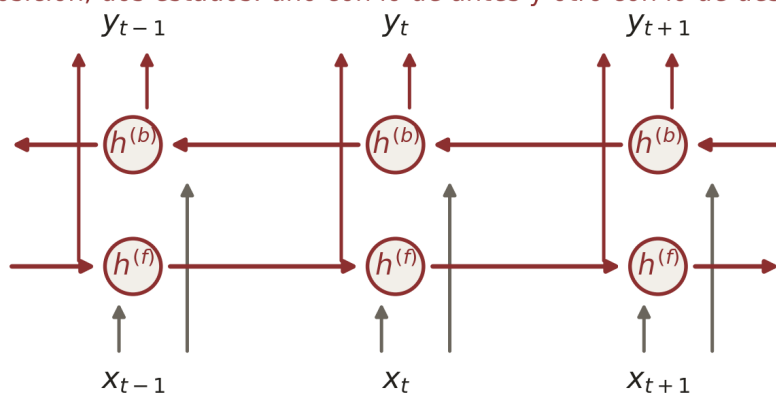
$$h_t^{(b)} = g(W^{(b)}x_t + U^{(b)}h_{t+1}^{(b)})$$

lee hacia atrás

$$y_t = \text{softmax}(V^{(f)}h_t^{(f)} + V^{(b)}h_t^{(b)})$$

el cabezal ve las dos

una posición, dos estados: uno con lo de antes y otro con lo de después



la cadena de abajo avanza y la de arriba retrocede; las dos entregan al mismo cabezal

Dos juegos de matrices

W, U y V por cada sentido: son dos redes, no una.

Anterior contra siguiente

La de ida toma el estado de $t-1$; la de vuelta, el de $t+1$.

El cabezal ve las dos

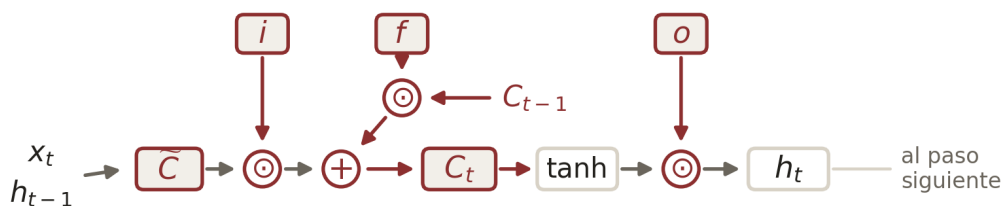
Suma los dos aportes y recién ahí viene la softmax.

Cuando una sola compuerta no alcanza

En secuencias largas el estado se rehace tantas veces que lo del principio se pierde. Las compuertas existen para eso.

$$\begin{aligned}\tilde{C}_t &= \tanh(W_c x_t + U_c h_{t-1}) && \text{lo nuevo que se propone guardar} \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + V_i C_{t-1}) && \text{cuánto de eso entra} \\ f_t &= \sigma(W_f x_t + U_f h_{t-1} + V_f C_{t-1}) && \text{cuánto de lo viejo se conserva} \\ \mathbf{C}_t &= f_t \odot \mathbf{C}_{t-1} + i_t \odot \tilde{C}_t && \text{el contexto: se atenúa y se suma} \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + V_o \mathbf{C}_t) && \text{cuánto del contexto sale} \\ h_t &= o_t \odot \tanh(\mathbf{C}_t) && \text{el estado que ve el resto de la red}\end{aligned}$$

las tres compuertas son la misma capa con sigmoide; lo que cambia es qué filtran



el contexto cruza la celda sin pasar por ninguna capa: solo lo multiplican y le suman
por ese camino la información viaja lejos sin rehacerse

El problema es rehacer

Cada paso reescribe el estado: lo viejo se pierde.

LSTM y GRU

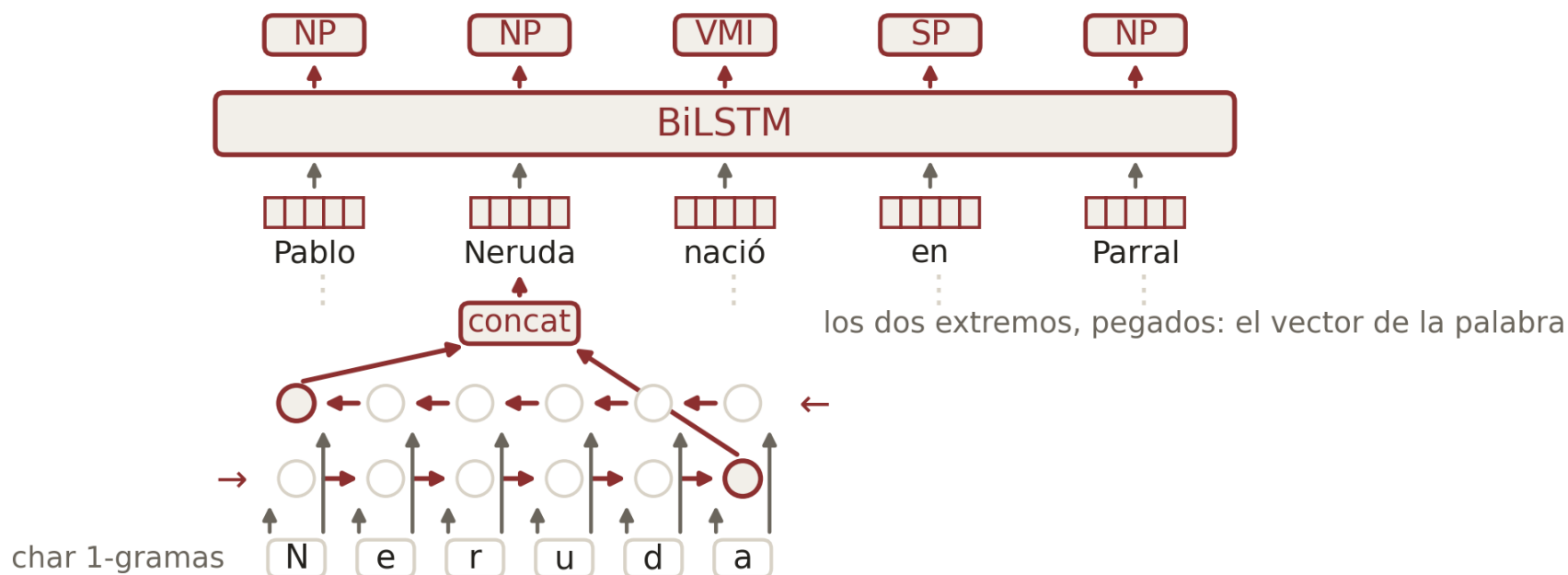
Tres compuertas la LSTM; dos la GRU.

El círculo con punto es el producto elemento a elemento, coordenada por coordenada.

De los caracteres a la palabra

Con un tokenizador que entregue las palabras, sus vectores se pueden armar desde los caracteres y pasarlos al cabezal.

el tokenizador da las palabras; los caracteres dan sus vectores



sin tabla de palabras: cualquier palabra tiene vector, la haya visto o no

El vector de una palabra, letra por letra

El char 1-grama por dentro: una bidireccional sobre embeddings de carácter, y de todo lo que produce se guardan dos vectores.

$$e_j = E_c [c_j]$$

un vector por carácter

$$h_j^{(f)} = \text{LSTM}^{(f)}(e_j, h_{j-1}^{(f)})$$

de la N a la a

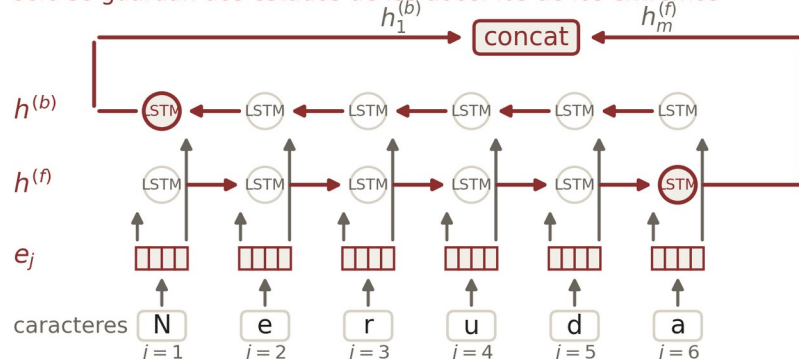
$$h_j^{(b)} = \text{LSTM}^{(b)}(e_j, h_{j+1}^{(b)})$$

de la a a la N

$$\mathbf{v}_{\text{Neruda}} = [h_m^{(f)} ; h_1^{(b)}]$$

los dos extremos, pegados

solo se guardan dos estados de los doce: los de los extremos



los caracteres también tienen embeddings; su tabla es de decenas de filas, no de decenas de miles

Los caracteres tienen tabla

Decenas de filas, no decenas de miles.

Sobreviven dos estados

El último de ida y el primero de vuelta, pegados.